

Assignment 3

Sanjitha S, 2023CS50734

April 11, 2026

1 Parallelization Strategy

The implementation uses a fully decentralized **Scatter-Then-Compute** architecture combined with **Ring-Based Work Stealing** to ensure optimal load balancing and minimal network overhead.

- **Initial Scatter (Kickstart):** Rank 0 performs a BFS on the search tree to generate independent subproblems (chunks). These chunks are dealt round-robin to all processes using non-blocking `MPI_Isend` to kickstart computation.
- **Asynchronous Bound Updates:** When a process finds a new local maximum profit (P_{max}), it immediately broadcasts it to all other peers. To prevent Eager Buffer overflows and deadlocks, this is done using non-blocking "fire-and-forget" `MPI_Isend` calls.
- **Ring-Based Work Stealing:** Instead of a random-polling model that spams the network with $O(P)$ messages, idle ranks pass a `NEED_WORK` token to `rank + 1`. If a peer has work, it donates half its stack; if not, it forwards the token. A token completing a full circle indicates an empty cluster.
- **Distributed Termination:** Termination is managed via Dijkstra's Dual-Pass Token Ring. To prevent segmentation faults from ghost messages during teardown, a Distributed Quiescence Protocol tracks global `msgs_sent` and `msgs_recv` using `MPI_Allreduce`, mathematically proving the network is empty before calling `MPI_Finalize`.

2 Sequential & Data-Structure Optimizations

To strictly adhere to the mandated Branch-and-Bound algorithms while maximizing throughput, the following memory optimizations were applied:

- **Bitset Adjacency:** The graph is packed into 64-bit integers. One 64-byte cache line covers 512 neighbor bits, drastically reducing L1 cache misses.
- **Bitwise Color Bounding:** The structural graph coloring bound uses bitwise `AND` for conflict detection, reducing complexity from $O(N^2)$ to $O(N \times Colors \times \lceil N/64 \rceil)$.

- **Static Memory Buffers:** The `g_color_buf` is pre-allocated and reused across recursive calls to eliminate heap allocations.
- **Zero-Allocation Knapsack:** The fractional knapsack bound relies on a pre-sorted global efficiency array, avoiding runtime sorting.

3 Performance Analysis

MPI Ranks	Seq Time (s)	MPI Time (s)	Speedup
1	106.468	26.615	4.00x
2	106.468	13.880	7.67x
4	106.468	7.608	13.99x
8	106.468	5.247	20.28x
12	106.468	4.266	24.95x
16	106.468	4.481	23.75x

Table 1: Execution Time and Speedup vs. Number of Processes

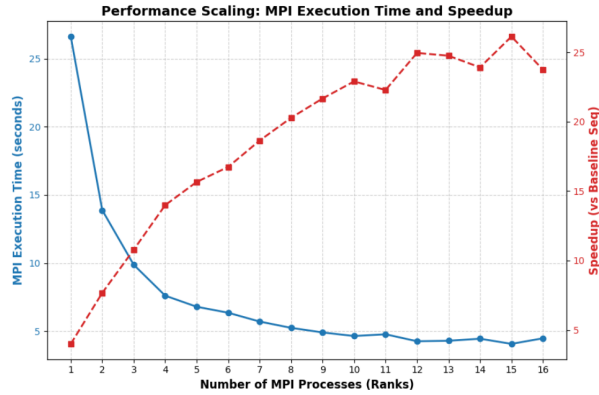


Figure 1: Execution Time and Speedup Scaling on input graph